



MÓDULO 2 · T06

Stack Navigator con React Navigation + TypeScript

 *Navegar entre pantallas*

Carlos Rodríguez Contreras · GitHub: [karrocon](#)

¿Qué aprenderás hoy?

En esta sesión dominarás la navegación entre pantallas con React Navigation. Al terminar, serás capaz de configurar un Stack Navigator completamente tipado y pasar parámetros entre pantallas de forma segura.

1

Instalar React Navigation

Configurar `@react-navigation/native` y `native-stack` en un proyecto Expo.

2

Stack Navigator tipado

Crear un `StackParamList` con TypeScript para tipar pantallas y parámetros.

3

Navegar entre pantallas

Usar `navigation.navigate()` para moverse de una pantalla a otra.

4

Pasar parámetros

Enviar y recibir datos entre pantallas con `route.params` de forma tipada.

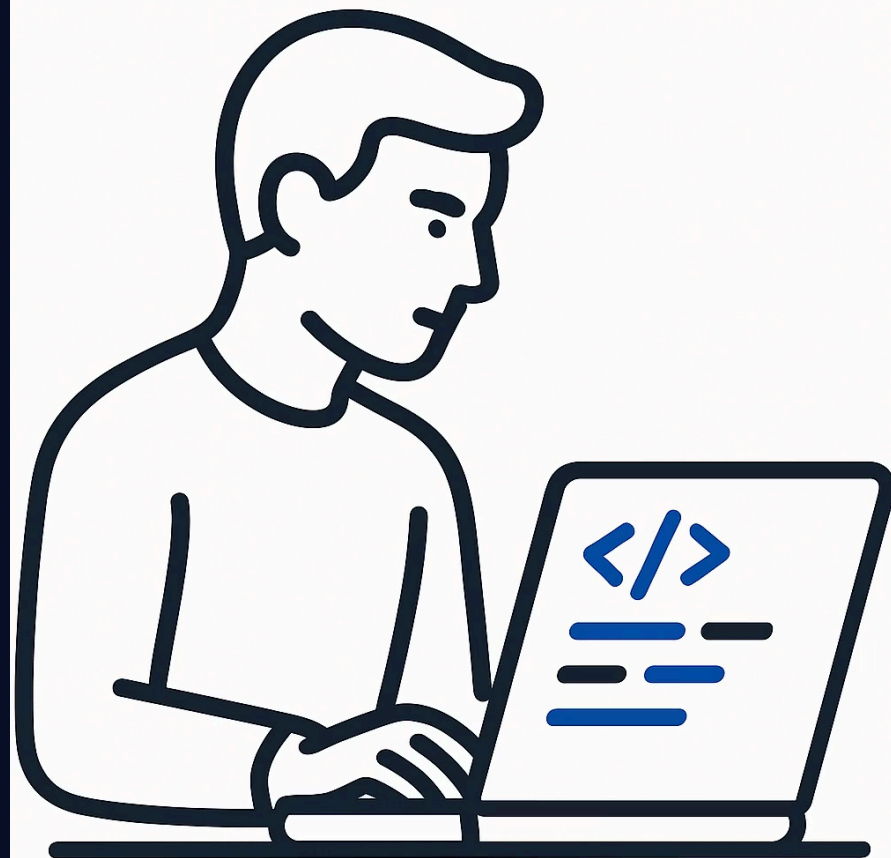
Instalación

Antes de crear el navigator, es necesario instalar las dependencias de React Navigation y los módulos nativos requeridos por Expo. El comando `npx expo install` garantiza versiones compatibles con tu SDK.

```
npm install @react-navigation/native \
  @react-navigation/native-stack
```

```
npx expo install react-native-screens \
  react-native-safe-area-context
```

❗ Usa siempre `npx expo install` para las dependencias nativas – elige automáticamente la versión compatible con tu versión de Expo SDK.



Estructura básica

¿Qué necesito?

Todo proyecto con React Navigation necesita un `NavigationContainer` en la raíz que envuelva al navigator. Dentro se declaran las pantallas disponibles con `Stack.Screen`.

TypeScript detectará en tiempo de compilación si navegas a una pantalla que no existe o si te faltan parámetros obligatorios.

- ✓ Sin tocar el código de cada pantalla, ya tienes navegación funcional entre `Home` y `Detalle`.

App.tsx básico

```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

const Stack = createNativeStackNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen
          name="Home"
          component={HomeScreen} />
        <Stack.Screen
          name="Detalle"
          component={DetalleScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

TypeScript: tipado del Stack

El tipo `StackParamList` define el contrato de navegación: qué pantallas existen y qué parámetros espera cada una. Si una pantalla no necesita parámetros se anota con `undefined`.

```
type StackParamList = {  
  Home: undefined;    // sin parámetros  
  Detalle: {  
    nombre: string;  
    id: number;  
  };                  // con parámetros  
};  
  
const Stack = createNativeStackNavigator<StackParamList>();
```

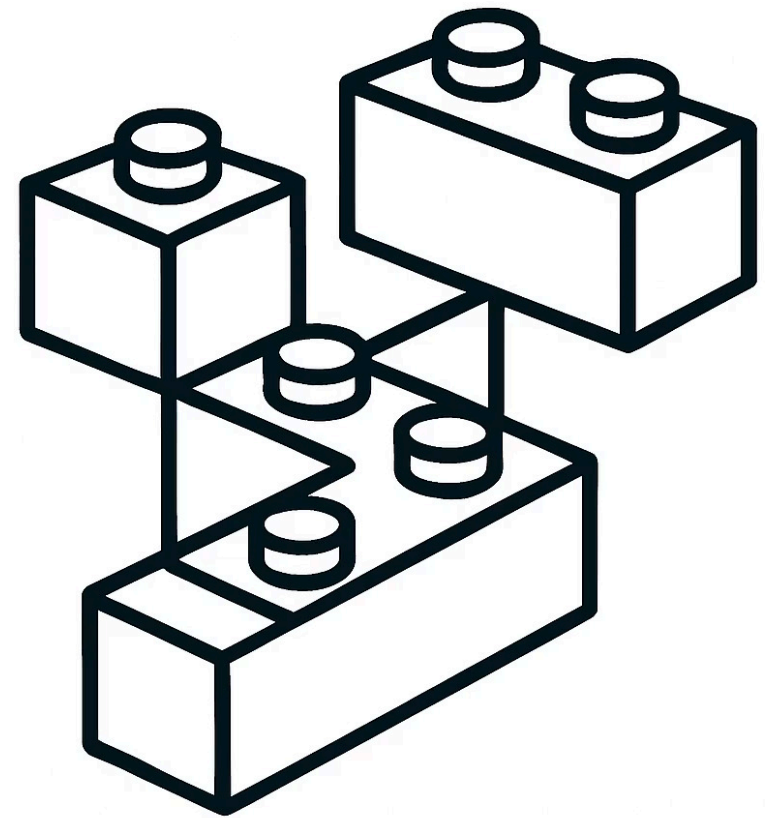
❗ TypeScript te avisa si intentas navegar a una pantalla inexistente o si te faltan parámetros requeridos – sin esperar a ejecutar la app

Navegación y parámetros

Para navegar a otra pantalla se usa `navigation.navigate()` pasando el nombre de la ruta y los parámetros. En la pantalla destino, `route.params` expone los valores enviados.

```
// Enviar parámetros desde HomeScreen
function HomeScreen({ navigation }) {
  return (
    <Button
      title="Ver Pikachu"
      onPress={() => navigation.navigate('Detalle', {
        nombre: 'pikachu',
        id: 25,
      })}
    />
  );
}
```

```
// Recibir parámetros en DetalleScreen
function DetalleScreen({ route }) {
  const { nombre, id } = route.params;
  return <Text>{nombre} — #{id}</Text>;
}
```



Props tipadas con NativeStackScreenProps

En lugar de dejar las props sin tipar, `NativeStackScreenProps` conecta el tipo de la pantalla con el `StackParamList`, proporcionando autocompletado completo tanto en `route.params` como en los métodos de `navigation`.

```
import type { NativeStackScreenProps } from '@react-navigation/native-stack';
```

```
type DetalleProps = NativeStackScreenProps<StackParamList, 'Detalle'>;
```

```
function DetalleScreen({  
  route,  
  navigation,  
}: DetalleProps) {  
  const { nombre, id } = route.params; // Tipado completo  
  return (  
    <>  
      <Text>{nombre}</Text>  
      <Button title="Volver" onPress={() => navigation.goBack()} />  
    </>  
  );  
}
```

Personalizar la cabecera

Estilos globales

```
<Stack.Navigator
  screenOptions={{
    headerStyle: {
      backgroundColor: '#EF5350',
    },
    headerTintColor: '#fff',
    headerTitleStyle: {
      fontWeight: '700',
    },
  }}
>
```

Con `screenOptions` aplicas un estilo global a todas las cabeceras del navigator de una sola vez.

Título dinámico por pantalla

```
<Stack.Screen
  name="Pokedex"
  component={PokedexScreen}
  options={{ title: 'Mi Pokédex' }}
/>
<Stack.Screen
  name="Detalle"
  component={DetalleScreen}
  options={({ route }) => ({
    title: route.params.nombre,
  })}
/>
```

❗ La prop `options` acepta tanto un objeto estático como una función que recibe `route` y `navigation` – ideal para títulos dinámicos basados en parámetros.

PokeApp — navegación tipada

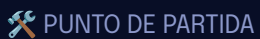
Este es el fichero de tipos de navegación tal y como aparece en la aplicación de ejemplo. Las constantes de ruta evitan errores tipográficos y permiten renombrar rutas desde un único lugar.

```
// src/types/navigation.ts
export type PokedexStackParamList = {
  [POKEDEX]: undefined;
  [DETALLE]: { pokemon: Pokemon };
};

// En PokedexScreen:
navigation.navigate(DETALLE, { pokemon });

// En DetallePokemonScreen:
const { pokemon } = route.params;
```

📄 **Buena práctica:** define las constantes de ruta y el ParamList en un fichero dedicado `src/types/navigation.ts` para que todas las pantallas las importen desde un único lugar.



PUNTO DE PARTIDA

Tour starter/App.tsx

Este es el fichero que encontrarás en la carpeta `starter/`. Contiene la estructura básica con tres puntos marcados como `TODO` que deberás completar durante el ejercicio. El tipo solo incluye `Home` y la navegación a `Detalle` está pendiente.

```
type StackParamList = {  
  Home: undefined;  
  // TODO: añade Detalle con parámetros  
};  
  
const Stack = createNativeStackNavigator<StackParamList>();  
  
// TODO: navega a Detalle desde HomeScreen  
// TODO: muestra route.params.nombre en DetalleScreen
```

⚠ No modifiques los estilos ni la estructura del `NavigationContainer` – céntrate en los tres `TODO` marcados arriba.

Ejercicio T06

Pon en práctica todo lo aprendido. Abre el proyecto en tu editor y sigue los pasos en orden – cada uno construye sobre el anterior. El bonus es opcional pero muy recomendable para reforzar el uso de imágenes con parámetros de navegación.

1 Abre el fichero de inicio

Navega a `starter/App.tsx` en tu editor.

2 Amplía el ParamList

Añade `Detalle: { nombre: string; id: number }` al tipo `StackParamList`.

3 Navega desde HomeScreen

En `HomeScreen`, navega a `Detalle` al pulsar un botón pasando `nombre` e `id`.

4 Muestra los parámetros

En `DetalleScreen`, extrae y muestra `route.params.nombre` en un `Text`.

5 Bonus: artwork oficial

Muestra la imagen `official-artwork` del Pokémon usando `route.params.id` como parte de la URL de la PokéAPI.

Resumen de la sesión

Estos son los conceptos clave que has trabajado hoy. Dominarlos te permitirá crear flujos de navegación robustos, completamente tipados y fáciles de mantener en cualquier app React Native.

Concepto	API clave	Para qué sirve
NavigationContainer	<code><NavigationContainer></code>	Wrapper obligatorio en la raíz
createNativeStackNavigator	<code>createNativeStackNavigator<T>()</code>	Crea un Stack tipado
ParamList	<code>type StackParamList = {...}</code>	Tipado de pantallas + parámetros
Navegar	<code>navigation.navigate(name, params)</code>	Ir a una pantalla pasando datos
Recibir params	<code>route.params</code>	Leer parámetros en la pantalla destino
Volver atrás	<code>navigation.goBack()</code>	Desapilar la pantalla actual
screenOptions	<code>screenOptions={{...}}</code>	Estilos globales del header

👉 ¡Enhorabuena! Ahora tienes las herramientas para construir navegación entre pantallas tipada, segura y personalizada en cualquier proyecto React Native. 🚀